Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez

Theory

Part 1. Questions

1. What is the difference between Breadth-First Search and Uniform Cost Search in a domain where the cost of each action is 1?

When action costs are all equal to 1, Breadth-First Search and Uniform Cost Search have the same behavior in terms of node expansion order.
In BFS, the search explores layer by layer, so depth equals the number of steps. Since each step costs 1, depth is also equal to the total path cost. UCS expands nodes based on the lowest accumulated cost. Therefore, UCS ends up picking nodes in the exact same order as BFS, the difference is that in this case, BFS is more efficient.

2. Suppose that h1 and h2 are admissible heuristics (used in A*). Which of the following are also admissible? Justify your answers.

(a) (h1 + h2)/2: Admissible because in case $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$ the average of both numbers are still $\leq h^*(n)$

Example: $\frac{6+8}{2} = 7$, where $7 < h^*(n)$

(b) 2h1 Doubling an admissible heuristic can cause it to overestimate $h^*(n)$

Example: $2(6) = 12$, where $12 > h^*(n)$

(c) max(h1, h2) Since both $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$, the larger of the two is still bounded above by $h^*(n)$

3. For each of the following search algorithms, determine whether they are complete and optimal. Give a quick explanation why.

• Breadth-First Search: is complete (for finite branching factors) and optimal only when steps costs are equal. It explores the search space layer by layer, so that is a guarantee that the shallowest goal (which matches the lowest-cost path under uniform costs) is found first.

• Depth-First Search: is neither complete nor optimal. By exploring a single branch to its maximum depth before backtracking, it can get trapped in infinite loops and simply returns the first goal path it finds, the cost does not matter.

• Uniform Cost Search: is complete and optimal as long as step costs are strictly positive. It expands nodes in order of their accumulated path cost, ensuring that the first time a goal node is selected for expansion, its cost is minimal.

Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez

• Iterative Deepening Search: is complete and optimal for uniform step costs by having depth limits, it combines the low memory usage of DFS with the layer-by-layer optimality of BFS.

• Bidirectionnal Search: is complete and optimal with uniform step costs. It runs two simultaneous searches, for example, one forward from the start and one backward from the goal, so this permits the meeting in the middle to find the shortest path faster.

• Greedy Best-First Search: is neither complete nor optimal. It is guided just by the heuristic, it makes local choices while ignoring accumulated path costs, making it vulnerable to loops

• A* Search: is complete and optimal when the heuristic is admissible, and step costs are positive. Evaluating nodes with f(n) = g(n) + h(n), it balances cost paid with estimated cost remaining to guarantee the minimum-cost path

Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez

Part 2. Comparisons of algorithms

Algorithm	Nodes Expanded	Steps
BFS	823	46
DFS	1,126	222
A*	231	58

Experimental setup: 3 dirt particles, seed 101
python3 run_lab.py --search search --seed 101 --dirt 3 --no-gui #change search with bfs,dfs,astar.

BFS

```
Created world:  20x20, 3 dirt particles
Maze type:  labyrinth
Random seed: 101
Running without GUI...
Initial state:
{
    'agent_position': (13, 18),
    'dirt_collected': 0,
    'remaining_dirt': 3,
    'is_terminated': False
}
Agent: planning from (13, 18) to Dirt(10, 5) - dirty
Agent: starting Breadth First Search Method (BFS)
       Needed 0.5 msec, PathLength: 27, NumExpNodes: 520
Agent: NO-OP Action
Agent: Vacuuming Dirt
Agent: planning from (10, 5) to Dirt(8, 3) - dirty
Agent: starting Breadth First Search Method (BFS)
       Needed 0.2 msec, PathLength: 5, NumExpNodes: 54
Agent: NO-OP Action
Agent: Vacuuming Dirt
Agent: planning from (8, 3) to Dirt(1, 6) - dirty
Agent: starting Breadth First Search Method (BFS)
       Needed 0.3 msec, PathLength: 11, NumExpNodes: 249
Agent: NO-OP Action
Agent: Vacuuming Dirt

Simulation completed after 46 steps
Final state:
{'agent_position': (1, 6), 'dirt_collected': 3, 'remaining_dirt': 0, 'is_terminated': True}
SUCCESS: All dirt cleaned!
```

DFS

Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez

```
Created world:  20x20, 3 dirt particles
Maze type:  labyrinth
Random seed: 101
Running without GUI...
Initial state:
{
    'agent_position': (13, 18),
    'dirt_collected': 0,
    'remaining_dirt': 3,
    'is_terminated': False
}
Agent: planning from (13, 18) to Dirt(10, 5) - dirty
Agent: starting Depth First Search Method (DFS)
        Needed 0.4 msec, PathLength: 77, NumExpNodes: 376
Agent: NO-OP Action
Agent: Vacuuming Dirt
Agent: planning from (10, 5) to Dirt(8, 3) - dirty
Agent: starting Depth First Search Method (DFS)
        Needed 0.5 msec, PathLength: 101, NumExpNodes: 620
Agent: NO-OP Action
Step 100: {'agent_position': (5, 9), 'dirt_collected': 1, 'remaining_dirt': 2,
'is_terminated': False}
Agent: Vacuuming Dirt
Agent: planning from (8, 3) to Dirt(1, 6) - dirty
Agent: starting Depth First Search Method (DFS)
        Needed 0.2 msec, PathLength: 41, NumExpNodes: 130
Agent: NO-OP Action
Step 200: {'agent_position': (4, 2), 'dirt_collected': 2, 'remaining_dirt': 1,
'is_terminated': False}
Agent: Vacuuming Dirt

Simulation completed after 222 steps
Final state:
{'agent_position': (1, 6), 'dirt_collected': 3, 'remaining_dirt': 0, 'is_terminated': True}
SUCCESS: All dirt cleaned!
```

A*

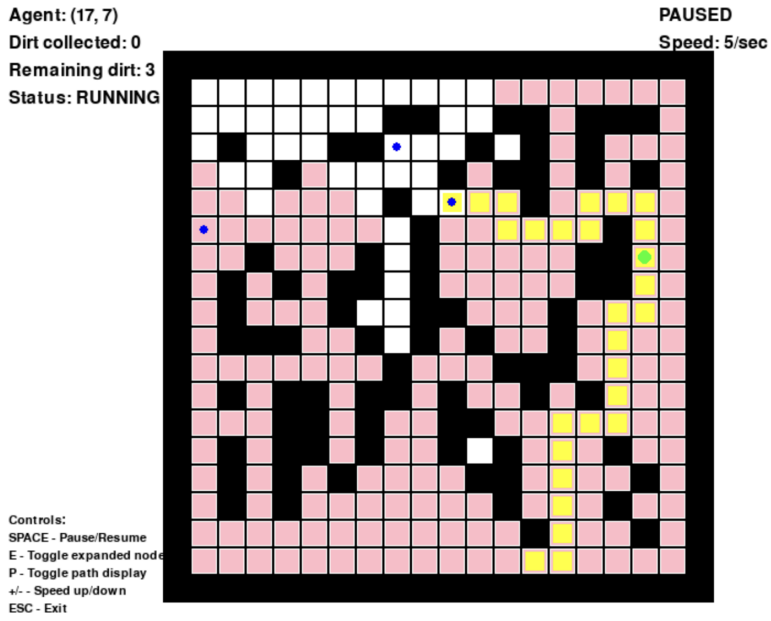
```
Created world:  20x20, 3 dirt particles
Maze type:  labyrinth
Random seed: 101
Running without GUI...
Initial state:
{
    'agent_position': (13, 18),
    'dirt_collected': 0,
    'remaining_dirt': 3,
    'is_terminated': False
}
Agent: planning from (13, 18) to Dirt(10, 5) - dirty
Agent: starting A*
        Needed 0.3 msec, PathLength: 39, NumExpNodes: 187
Agent: NO-OP Action
Agent: Vacuuming Dirt
Agent: planning from (10, 5) to Dirt(8, 3) - dirty
Agent: starting A*
        Needed 0.1 msec, PathLength: 5, NumExpNodes: 11
Agent: NO-OP Action
Agent: Vacuuming Dirt
Agent: planning from (8, 3) to Dirt(1, 6) - dirty
Agent: starting A*
        Needed 0.1 msec, PathLength: 11, NumExpNodes: 33
Agent: NO-OP Action
Agent: Vacuuming Dirt

Simulation completed after 58 steps
Final state:
{'agent_position': (1, 6), 'dirt_collected': 3, 'remaining_dirt': 0, 'is_terminated': True}
SUCCESS: All dirt cleaned!
```

Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez

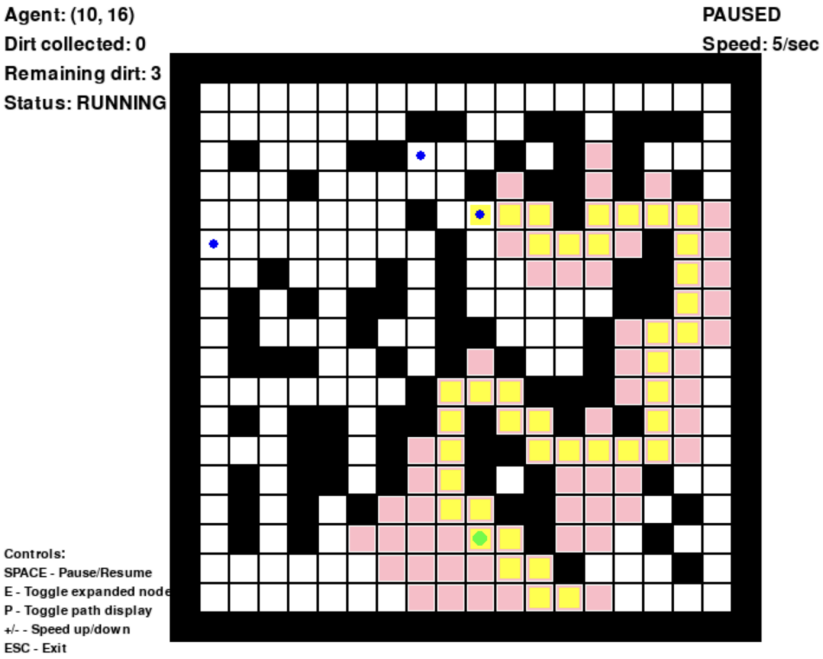
A* expanded significantly fewer nodes than BFS, this shows that the heuristic helped guide the search toward the goal more efficiently. However, A* produced a slightly longer path in terms of steps. This is due to the limitations of the Manhattan-distance heuristic in environments with obstacles: Manhattan distance estimates the direct distance to the goal but does not account for obstacles that may force the agent to temporarily move away from the goal. Therefore, the heuristic is not always sufficient to guide A* to the shortest path as effectively as BFS.
On the other hand, DFS proved to be the least efficient algorithm in terms of path quality. Due to its depth-first exploration, it explores long paths deep into the grid before backtracking, leading to a high number of visited nodes and producing a significantly longer, suboptimal final route compared to both BFS and A*.

BFS:



A*:

Vivian Morales Maldonado
Miguel Angel Gavito Gonzalez



DFS: